

8주차 스터디

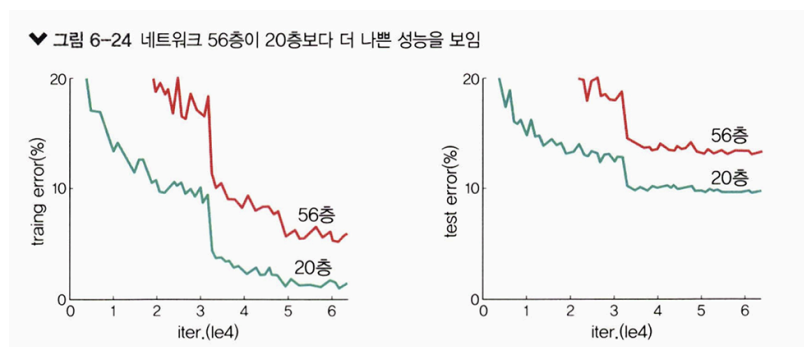
교재 : 딥러닝 파이토치 교과서

분량 : 6장 part 2

과제 기간 : ~2025-04-28

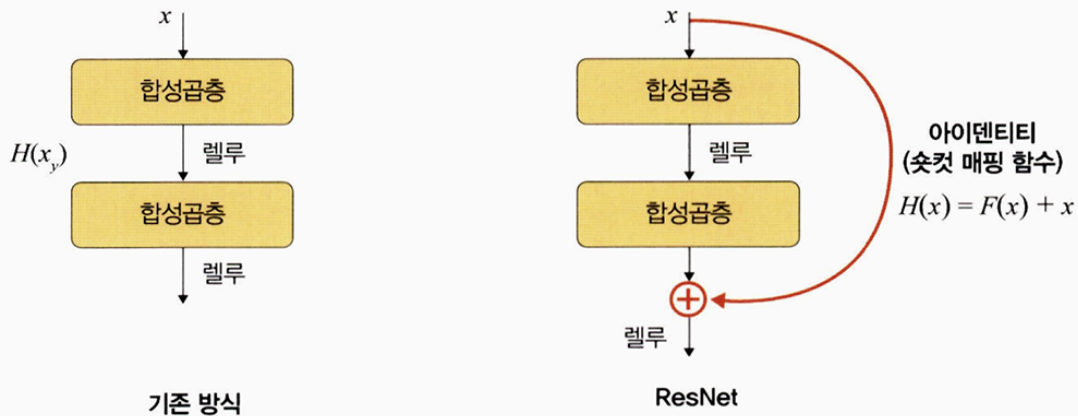
6.1.5 ResNet

- MS에서 개발한 알고리즘, Deep Residual Learning for Image Recognition 논문에서 발표
- ResNet의 핵심 : 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼 (Residual) 개념을 고안
- 신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정 단계에 다다르면 오히려 성능이 나빠진다
- 이 문제를 해결하기 위해 레지듀얼 블록(residual block)을 도입
- **레지듀얼 블록(residual block)** : 기울기가 잘 전파될 수 있도록 일종의 숏컷을 만들어 준다



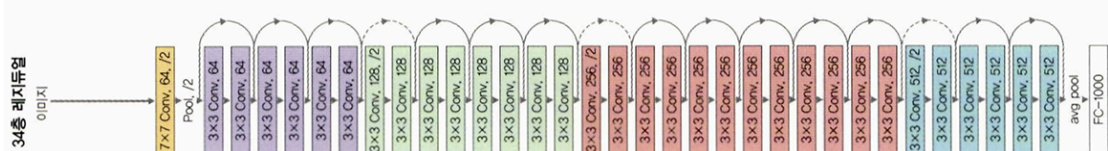
- GoogLeNet이 22개의 층으로 구성된 것에 비해 ResNet은 총 152개로 구성되어 기울기 소멸 문제가 발생할 수 있어서 아래 그림처럼 숏컷을 두어 기울기 소멸 문제를 방지했다

♥ 그림 6-25 ResNet 구조



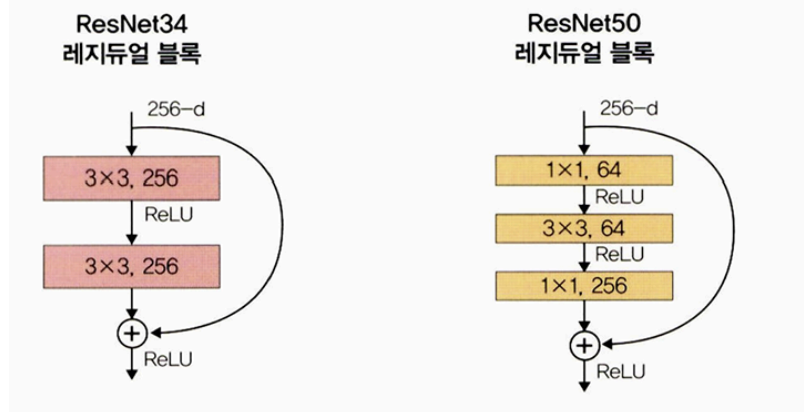
- **블록(block) 이란?** 계층의 묶음, 합성곱층을 하나의 블록으로 묶은 것
 - 아래 이미지에 색상별로 블록을 구분해두었는데 이렇게 묶인 계층들을 하나의 **레지듀얼 블록**이라고 한다
 - 레지듀얼 블록을 여러 개 쌓은 것을 **ResNet**이라고 한다

♥ 그림 6-26 ResNet 모델 전체 네트워크



- 계층을 계속 쌓아 늘리면 **파라미터 수**가 문제가 된다
- **계층이 깊어질수록 파라미터는 증가한다**
 - ex) ResNet34는 합성곱층이 34개와 16개의 블록으로 구성된다
 - 첫 번째 블록의 파라미터가 1152K라면 전체 파라미터 수는 2만 1282K이다. → 이처럼 계층의 깊이가 깊어질수록 파라미터는 무제한으로 커진다
 - 이 문제를 해결하기 위해 '**병목 블록(bottleneck block)**'이라는 것을 두었다
- 병목 블록을 두었을 때의 현상
 - ex) ResNet34와 ResNet50

♥ 그림 6-27 기본 블록과 병목 블록



- ResNet34는 기본 블록을 사용, ResNet50은 병목 블록을 사용한다
- 기본 블록의 파라미터 수 = 39.3216M, 병목 블록의 파라미터 수 = 6.9632M
→ 깊이가 깊어졌음에도 파라미터 수는 감소

Q. 어떻게 가능한 것일까?

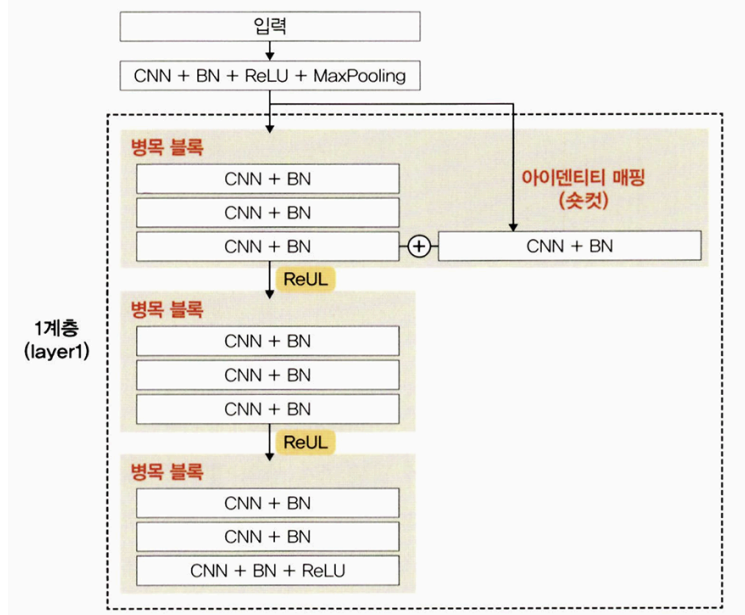
- 깊이가 깊어질수록 파라미터 수가 증가한다고 했는데, 병목 블록을 사용하면 파라미터 수가 감소하는 효과가 있다
- ResNet34와는 다르게 ResNet50에서는 **3x3 합성곱층 앞뒤로 1x1 합성곱층이 붙어 있다**
- 1x1 합성곱층의 채널 수를 조절하면서 **차원을 줄였다 늘리는 것이 가능해져 파라미터 수를 줄일 수 있다**

• 아이덴티티 매핑(identity mapping)에 대해 알아보기

*혹은 숏컷(shortcut), 스킵 연결(skip connection)이라고도 함

- 그림 6-27의 아래쪽에 + 기호 , 이 부분을 아이덴티티 매핑이라고 한다
- 아이덴티티 매핑 : 입력 x가 어떤 함수를 통과하더라도 다시 x라는 형태로 출력되도록 한다

▼ 그림 6-28 아이덴티티 매핑(숏컷)



○ 코드 예시 :

```
def forward(self,x) :
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

    if self.downsample is not None :
        i = self.downsample(i) ###다운샘플 적용

    x += i ###아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

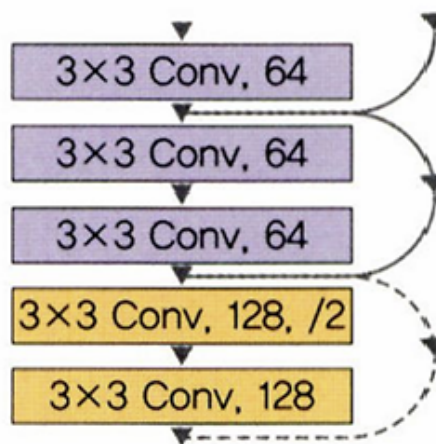
1. 입력 x 를 i 라는 변수에 저장
2. 입력 x 는 합성곱층을 통과하다가 마지막 x 에 i 를 더해 주었다
3. 예를 들어 x 가 (28,28,64)라고 가정하면, x 를 i 에 저장했기 때문에 i 는 (28,28,64)가 된다.
4. 그리고 합성곱층을 통과하면서 같은 형태를 더하기 때문에 최종 형태는 (28,28,64) 그대로가 된다

- **다운샘플(down sample)알아보기**

- 다운샘플은 특성 맵(feature map)크기를 줄이기 위한 것
- 풀링과 같은 역할을 한다

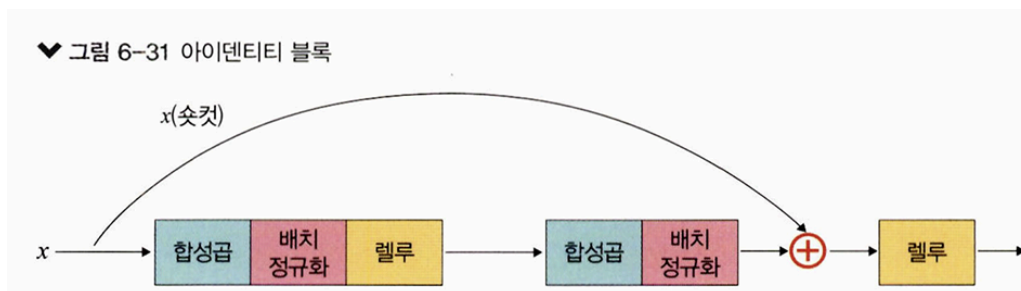
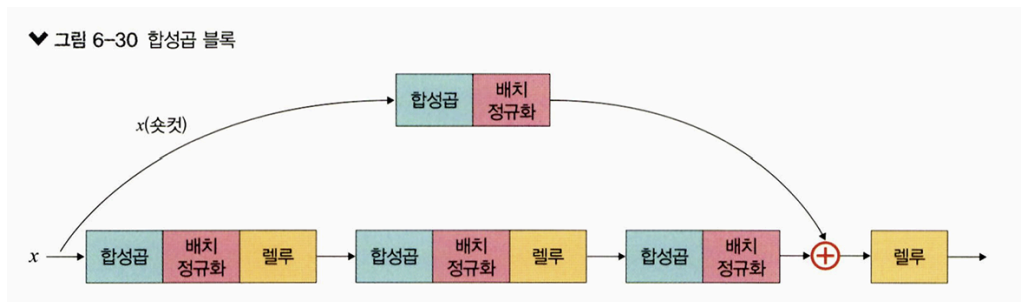
ex)

▼ 그림 6-29 ResNet 네트워크의 일부



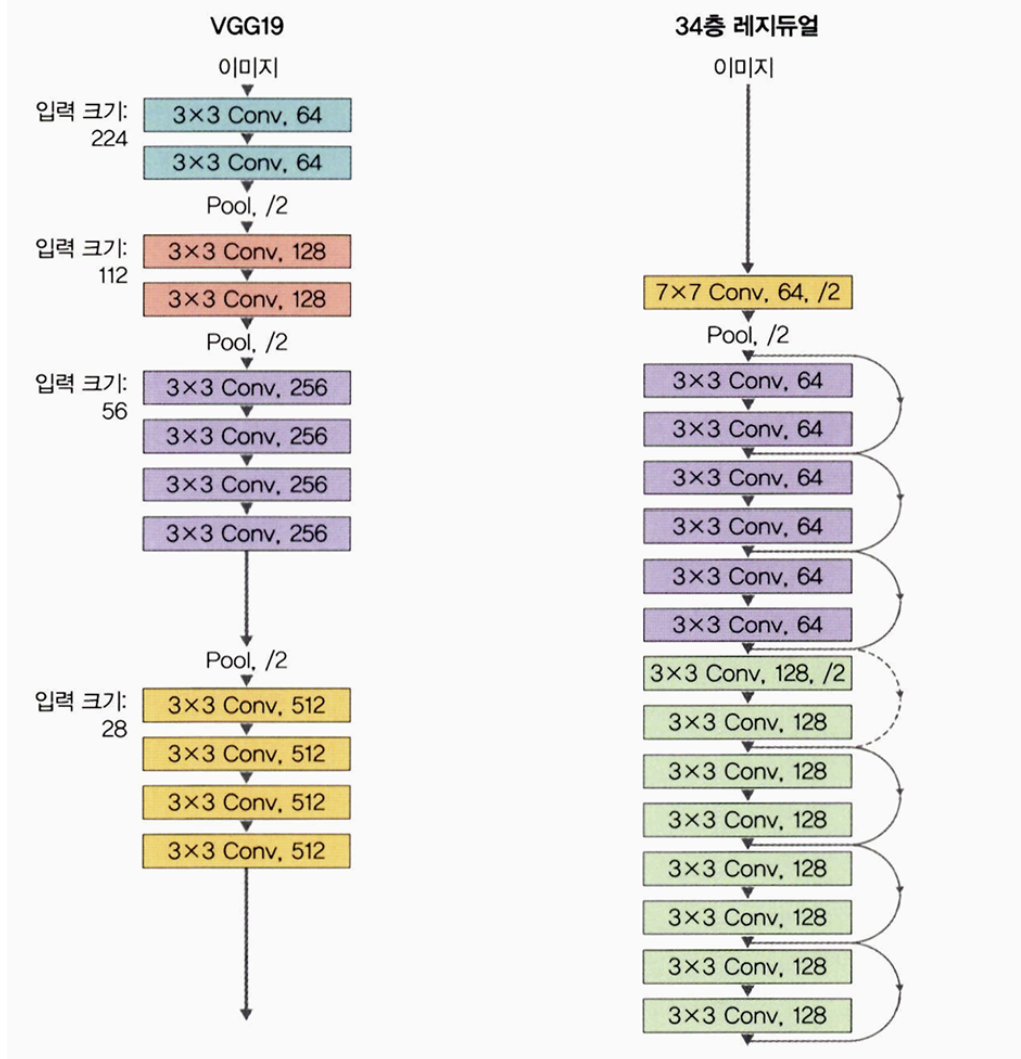
- 보라색 영역 : 첫 번째 블록에서 특성 맵의 형상이 (28,28,64)였다면 세 번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑까지 완료된 특성 맵의 형상도 (28,28,64)이다.
 - 노란색 영역 : 시작 지점에서는 채널 수가 128로 늘어나고 /2라는 것으로 보아 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어나 (14,14,128)로 바뀐다는 것을 알 수 있다
 - 즉 보라색과 노란색 형태가 다른데 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑을 할 수 없다
→ 아이덴티티에 대해 다운샘플이 필요하다
- 입출력의 형태를 같도록 맞춰 주기 위해서는 스트라이드 2를 가진 1x1 합성곱 계층을 하나 연결해 주면 된다
 - 이와 같이 입력과 출력이 같은 것을 **아이덴티티 블록**이라고 하며

- 입력 및 출력 차원이 동일하지 않고 입력의 차원을 출력에 맞추어 변경해야 하는 것을 **프로젝션 숏컷(projection shortcut)** 혹은 **합성곱 블록**이라고 한다



- ResNet은 기본적으로 VGG19 구조를 뼈대로 하며
- 합성곱층들을 추가해서 깊게 만든 후
- 숏컷들을 추가하는 것이다

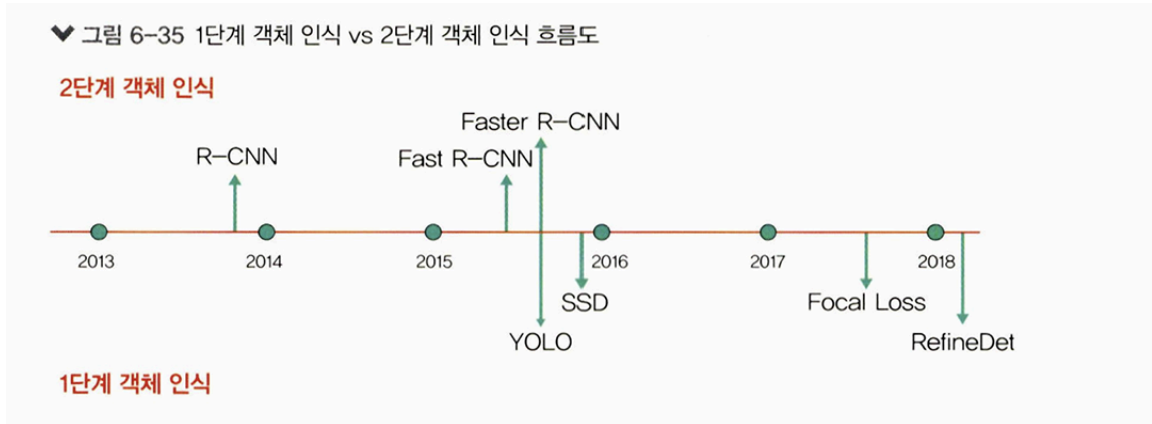
▼ 그림 6-32 VGG19와 ResNet 비교



6.2 객체 인식을 위한 신경망

- 객체 인식(object detection)은 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
- 즉, 객체 인식이란 이미지나 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스(bounding box)로 나타내는 위치 검출(localization) 문제를 다루는 분야
- 객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출
- 객체 인식 → 자율 주행 자동차, CCTV, 무인 점포 등에서 활용

- 딥러닝을 이용한 객체 인식 알고리즘은 1단계 객체 인식(1-stage detector)과 2단계 객체 인식(2-stage detector)으로 나뉜다



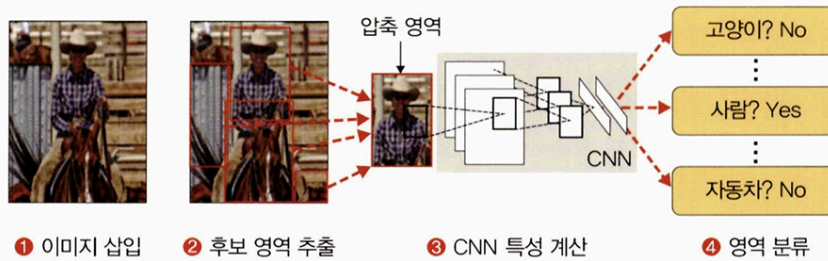
- 1단계 객체 인식 : 두 문제(분류와 위치 검출)을 동시에 행하는 방법
- 2단계 객체 인식 : 두 문제를 순차적으로 행하는 방법

1단계 객체 인식	2단계 객체 인식
분류, 위치 검출 동시에 시행	분류, 위치 검출 순차적으로 시행
빠르지만 정확도가 낮다	느리지만 정확도가 높다
YOLO 계열, SSD 계열 등	R-CNN 계열

6.2.1 R-CNN

- 현재의 선택적 탐색 알고리즘을 적용한 후보 영역을 사용하는 객체 인식 알고리즘
- **R-CNN** : 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해주는 후보 영역 알고리즘을 결합한 알고리즘
- R-CNN의 수행 과정

▼ 그림 6-36 R-CNN 학습 절차



1. 이미지를 입력으로 받습니다.
2. 2000개의 바운딩 박스(bounding box)를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고 (cropping), CNN 모델에 넣기 위해 같은 크기(227×227 픽셀)로 통일합니다(warping).
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용합니다.
4. 각각 분류를 진행하여 결과를 도출합니다.

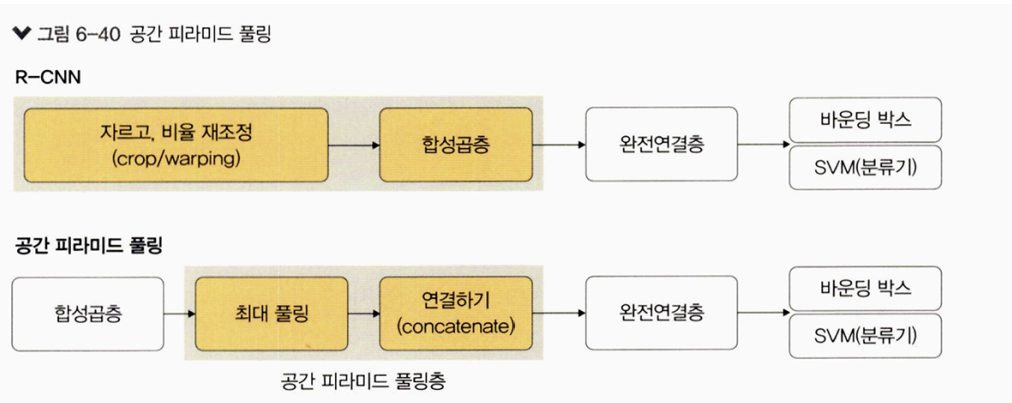
• R-CNN의 문제점

1. 앞서 언급한 세 단계의 복잡한 학습 과업
2. 긴 학습 시간과 대용량 저장 공간
3. 객체 검출 속도 문제

→ 이 문제들을 해결하기 위해서 Fast R-CNN이 생겼다

6.2.2 공간 피라미드 풀링

- 기존 CNN 구조들은 모두 완전연결층을 위해 입력 이미지를 고정해야 했다
- 그렇기 때문에 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나(crop)
- 비율을 조정(warp)해야 했다
- 하지만 이렇게 하면 물체의 일부분이 잘리거나 본래 생김새와 달라지는 문제가 생긴다
- 이러한 문제를 해결하고자 **공간 피라미드 풀링(spatial pyramid pooling)**을 도입했다



• 공간 피라미드 풀링이란?

- 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고
- 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법이다

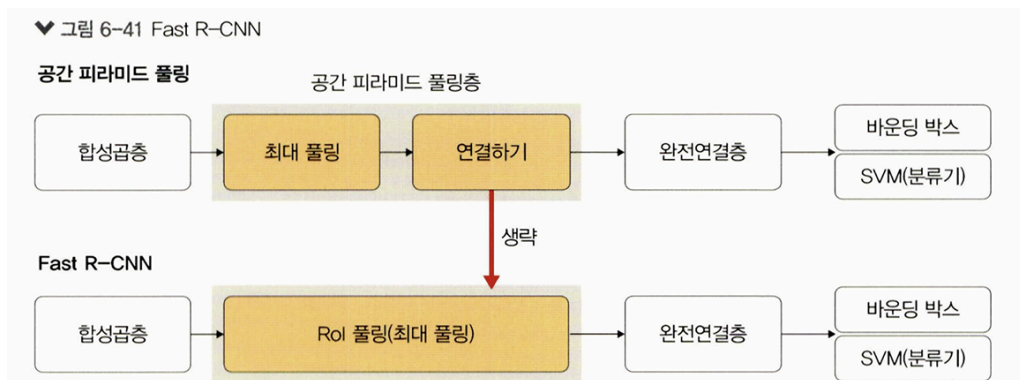
• 공간 피라미드 풀링의 장점

- 입력 이미지 크기를 조절하지 않고 합성곱층을 통과시키기 때문에 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있다
- 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있다

6.2.3 Fast R-CNN

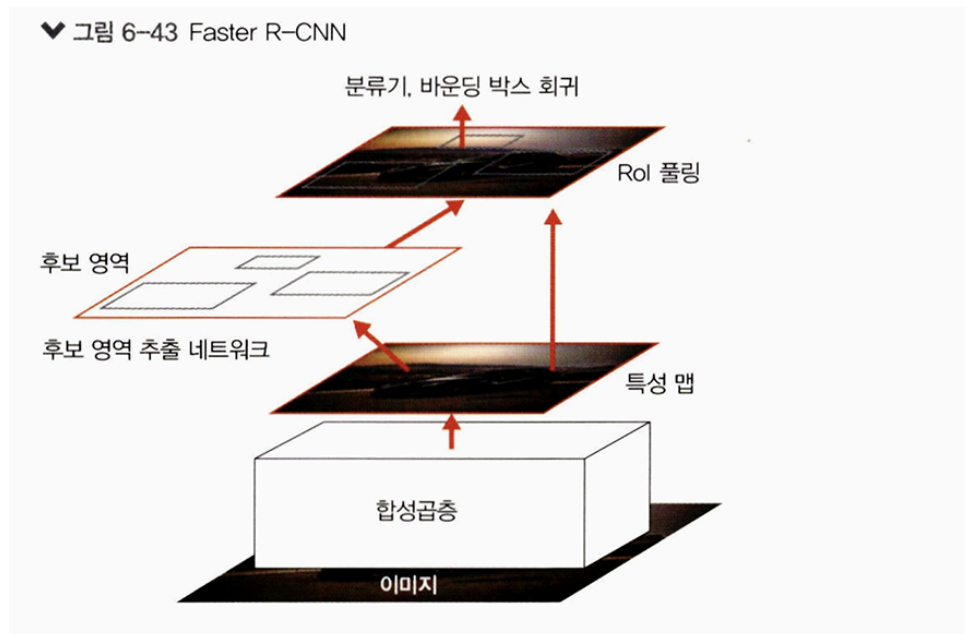
- **Fast R-CNN**은 R-CNN의 속도 문제를 개선하기 위해서 RoI풀링을 도입했다
- 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정한다

→ 바운딩 박스마다 CNN을 돌리는 시간을 단축할 수 있다



6.2.4 Faster R-CNN

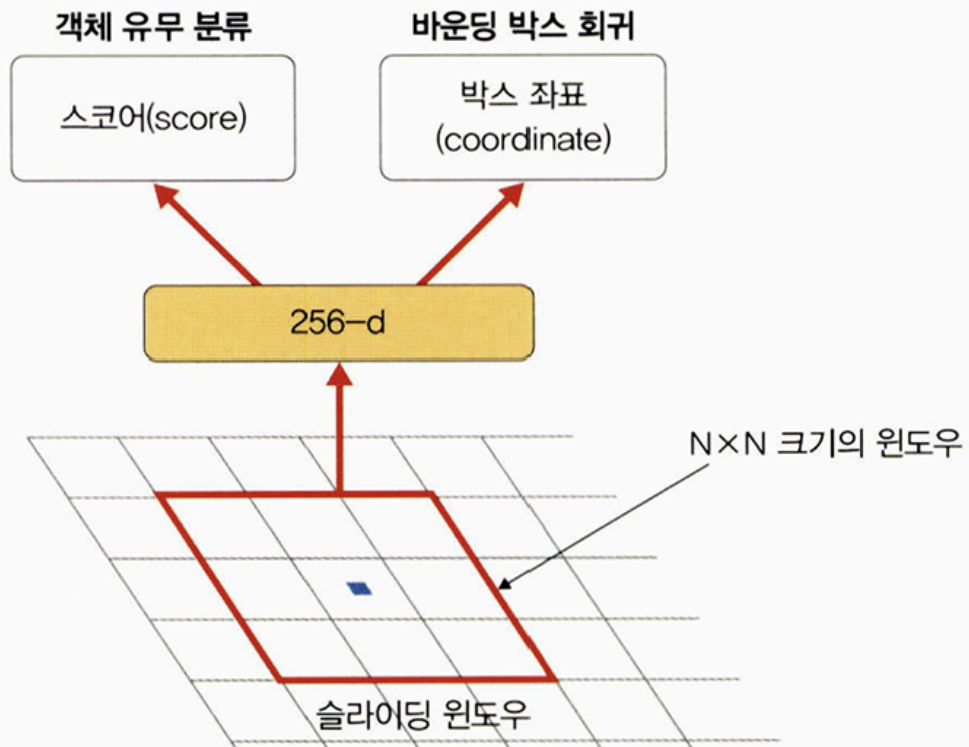
- 더욱 빠른 객체 인식을 수행하기 위한 네트워크
- 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계했다
- 기존 Fast R-CNN에 후보 영역 추출 네트워크를 추가한 것이 핵심
- 외부의 느린 선택적 탐색 대신 **내부의 빠른 RPN을 사용**한다
- RPN은 합성곱층 다음에 위치하고 그 뒤에 RoI 풀링과 분류기, 바운딩 박스 회귀가 위치한다



• 후보 영역 추출 네트워크

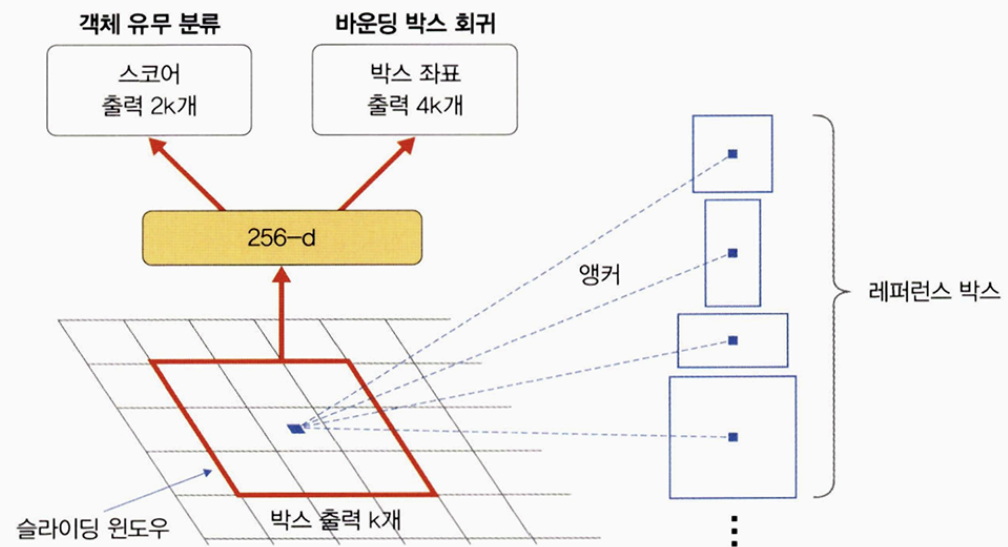
- 특성 맵 $N \times N$ 크기의 작은 윈도우 영역을 입력으로 받는다
- 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류를 수행하는 작은 네트워크를 생성한다
- 하나의 특성 맵에서 모든 영역에 대한 객체 존재 유무를 확인하기 위해서는 슬라이딩 윈도우 방식으로 앞서 설계한 작은 윈도우 영역을 이용해 객체를 탐색한다

▼ 그림 6-44 후보 영역 추출 네트워크



- 이미지에 존재하는 객체들의 크기와 비율이 다양해서 고정된 $N \times N$ 크기의 입력만으로 다양한 크기와 비율의 이미지를 수용하기 어렵다는 단점이 있다
→ 해결책: 레퍼런스 박스 k 개를 미리 정의한 뒤 각각의 슬라이딩 윈도우 위치마다 박스 k 개를 출력하도록 설계하는 “앵커 방식”을 이용한다

▼ 그림 6-45 앵커



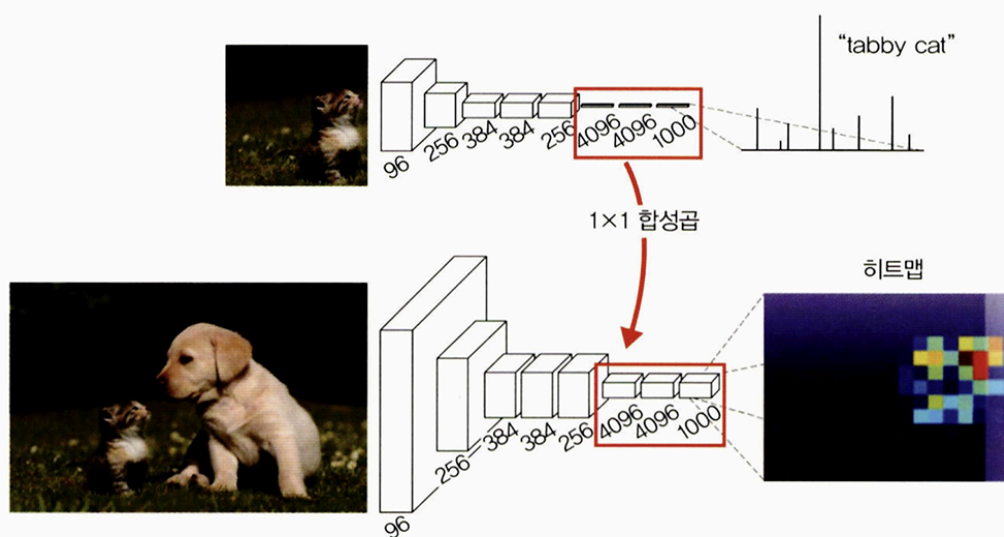
6.3 이미지 분할을 위한 신경망

- 이미지 분할 : 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것
- 이미지를 픽셀 단위로 분할해 이미지에 포함된 객체를 추출한다
- 대표적인 네트워크 : 완전 합성곱 네트워크, 합성곱& 역합성곱 네트워크, U-Net, PSPNet 등이 있다

6.3.1 완전 합성곱 네트워크

- 완전 합성곱 네트워크
 - 완전연결층을 1x1 합성곱으로 대체하는 것
 - 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크
 - 합성곱층으로 사용되기에 입력 이미지에 대한 크기 제약이 사라짐

▼ 그림 6-46 완전 합성곱 네트워크



6.3.2 합성곱 & 역합성곱 네트워크

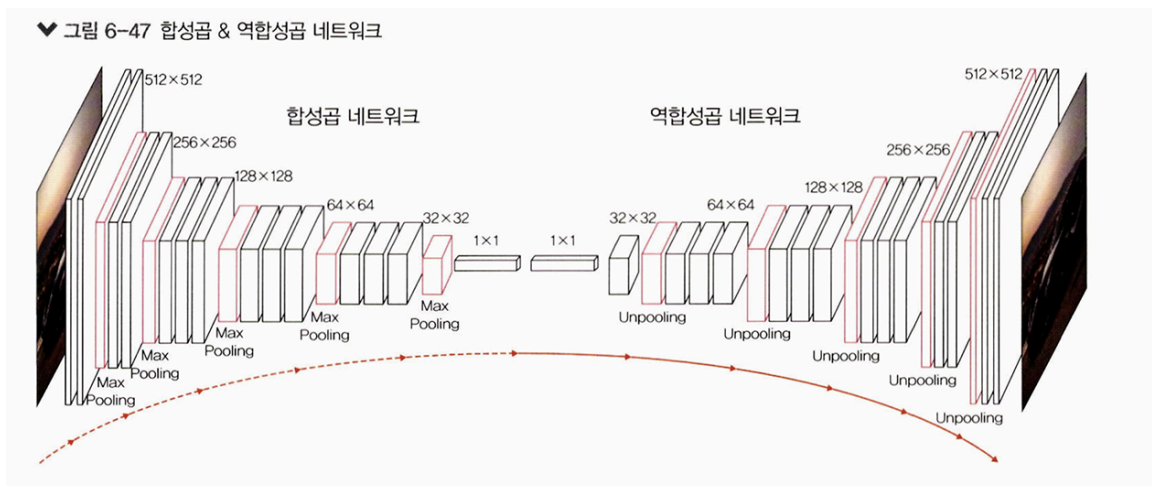
- 완전 합성곱 네트워크 단점

- 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아진다
- 낮아진 해상도를 복구하기 위해 업 샘플링 방식을 사용해서 이미지 세부 정보들을 잃어버리는 문제가 발생한다

→ 이 문제를 해결하기 위해 **역합성곱 네트워크 도입**

- 역합성곱

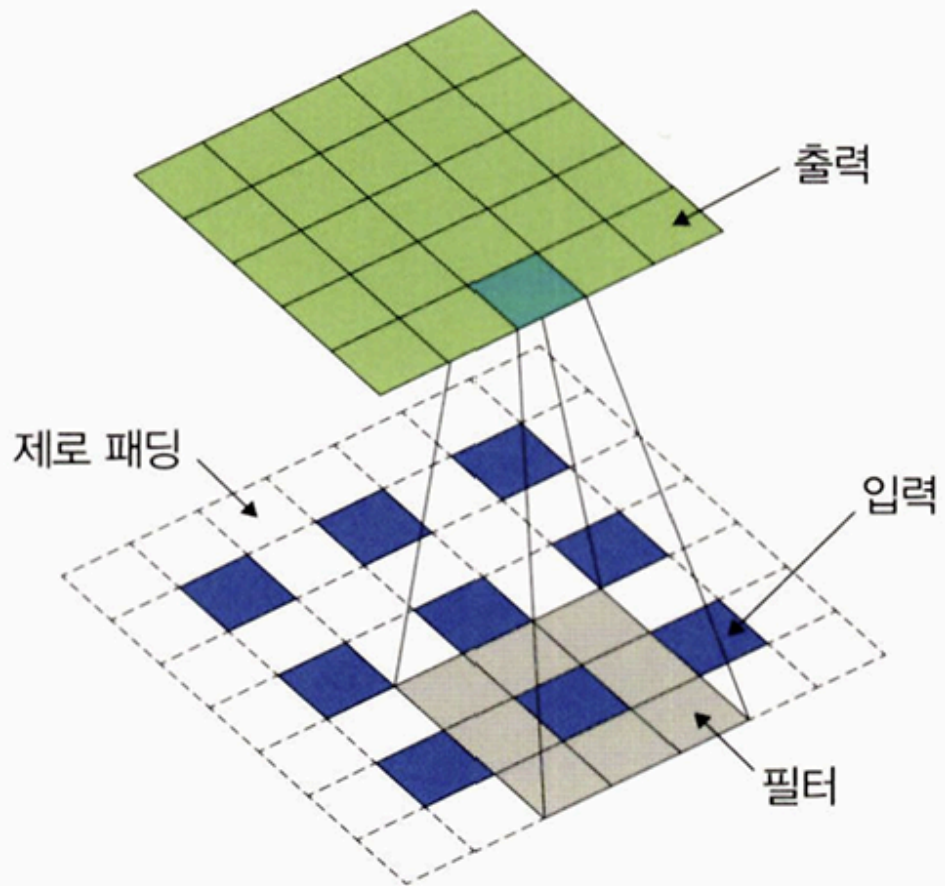
- CNN의 최종 출력 결과를 원래의 이미지와 같은 크기로 만들고 싶을 때 사용한다
- 시멘틱 분할 등에 활용, 업샘플링이라고 한다



- 역합성곱의 동작 방식 :

1. 각 픽셀 주위에 제로 패딩 추가
2. 패딩된 것에 합성곱 연산 수행

▼ 그림 6-48 역합성곱 진행 방식

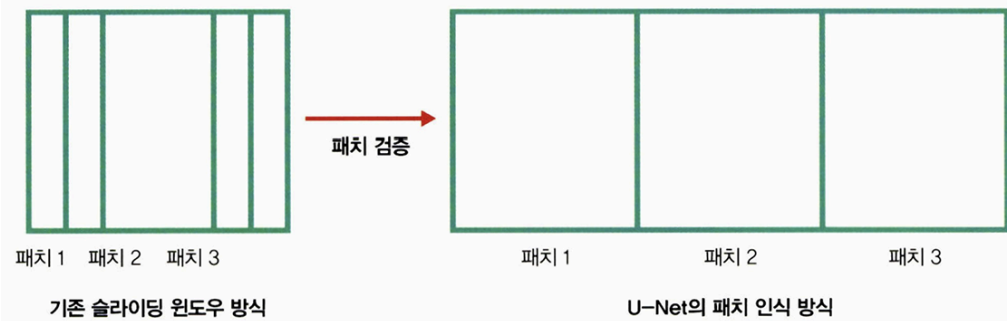


6.3.3 U-Net

- U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망이다
- 특징
 1. 속도가 빠름 : 이미 검증이 끝난 패치는 건너뛴

▼ 그림 6-49 슬라이딩 윈도우 방식



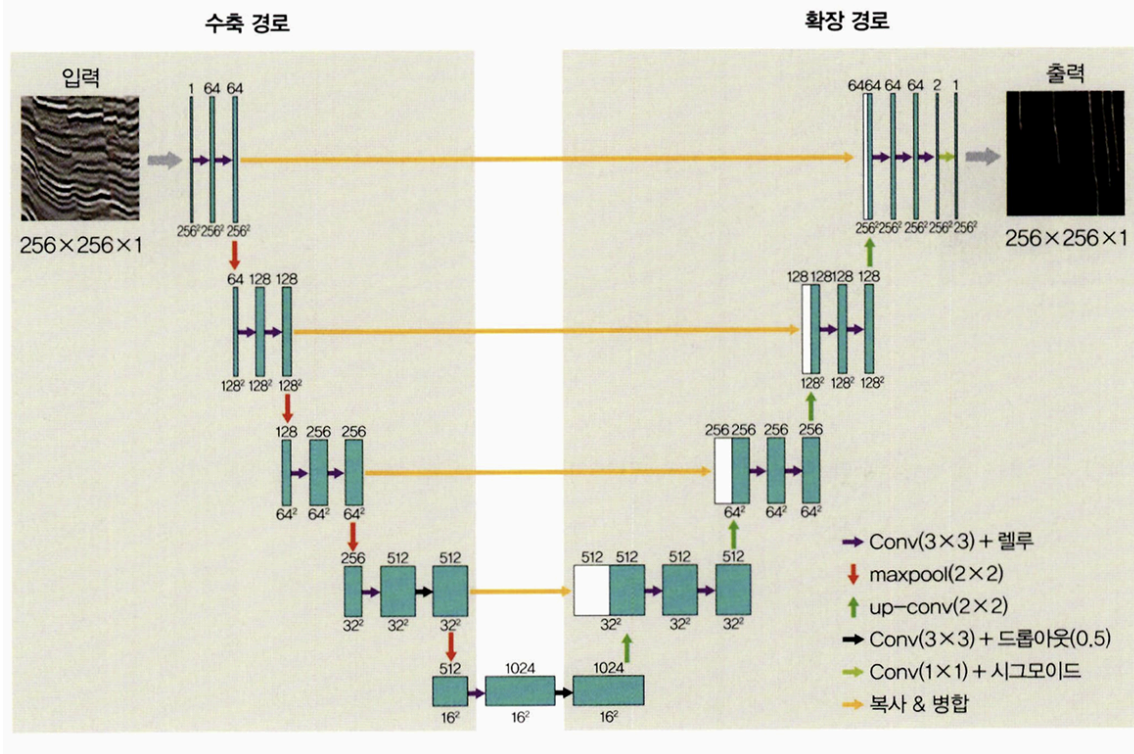
2. 트레이드오프에 빠지지 않음

• U-Net의 구조 :

- FCN 기반으로 구축, 수축 경로, 확장 경로로 구성
- 수축 경로 : 컨텍스트 포착
- 확장 경로 : 특성 맵 업샘플링, 수축 경로에서 포착한 특성맵의 컨텍스트와 결합해 정확한 지역화 수행
- U-Net은 3×3 합성곱이 주를 이루는데 각 합성곱 블록은 3×3 합성곱 두 개로 구성되어 있으며, 그 사이에 드롭아웃(dropout)이 있음
- 왼쪽 수축 경로에서의 블록은 3×3 합성곱 두 개로 구성된 것이 네 개가 있는 형태
- 각 블록은 최대 풀링을 이용해 크기를 줄이며 다음 블록으로 넘어감
- 오른쪽 확장 경로에서는 합성곱 블록에 up-conv라는 것을 앞에 붙였. 수축 과정에서 줄어든 크기를 다시 키워 가면서 합성곱 블록을 이용하는 형태

⇒ 크기가 다양한 이미지의 객체를 분할하기 위해 **크기가 다양한 특성 맵을 병합할 수 있도록 다운 샘플링, 업 샘플링 순서대로 반복하는 구조**

♥ 그림 6-51 U-Net

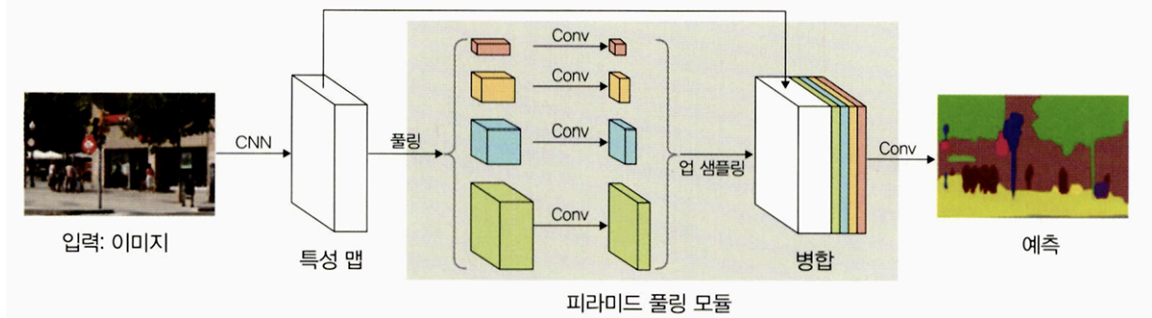


6.3.4 PSPNet

• PSPNet

- 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈 추가
- 훈련 과정
 1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링 시도 (1x1, 2x2, 3x3 크기로 수행) → 여기서 1x1이 가장 광범위한 정보 담음, 나머지는 서로 다른 영역들의 정보 담음
 2. 1x1 합성곱을 사용해 채널 수 조정. 풀링층 개수를 N이라고 할 때 출력 채널 수 = 입력 채널 수 / N
 3. 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링
 4. 원래 특성 맵, 1~3과정에서 생성한 새로운 특성 맵 병합

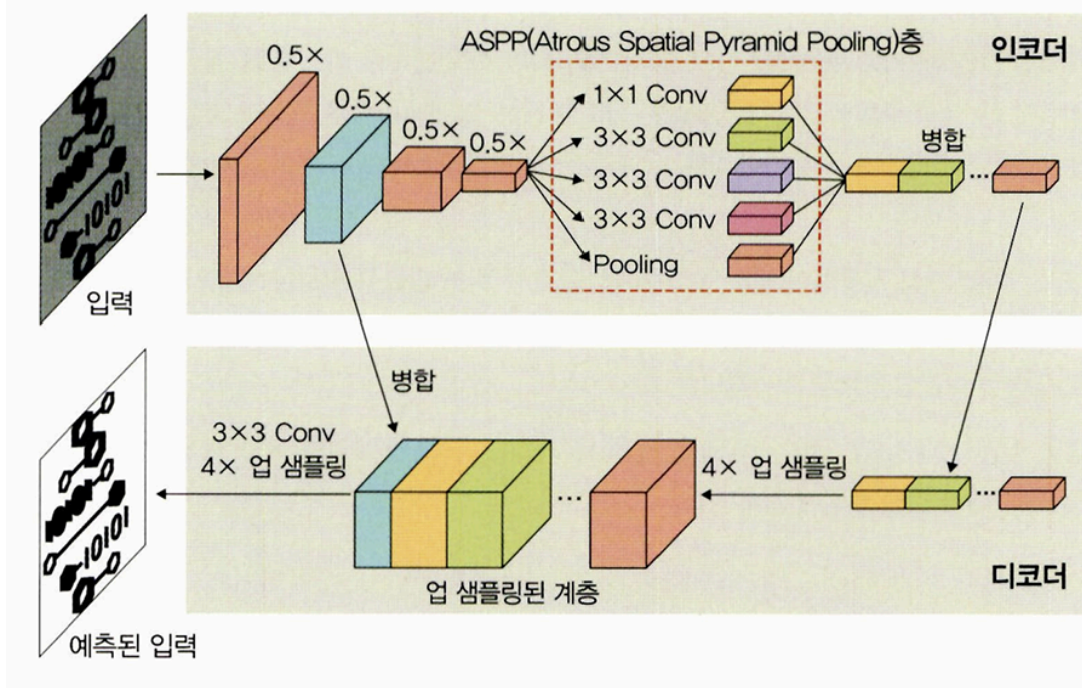
▼ 그림 6-52 PSPNet



6.3.5 DeepLabv3/DeepLabv3+

- DeepLabv3/DeepLabv3+
 - 완전연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크
 - 인코더, 디코더 구조를 가짐

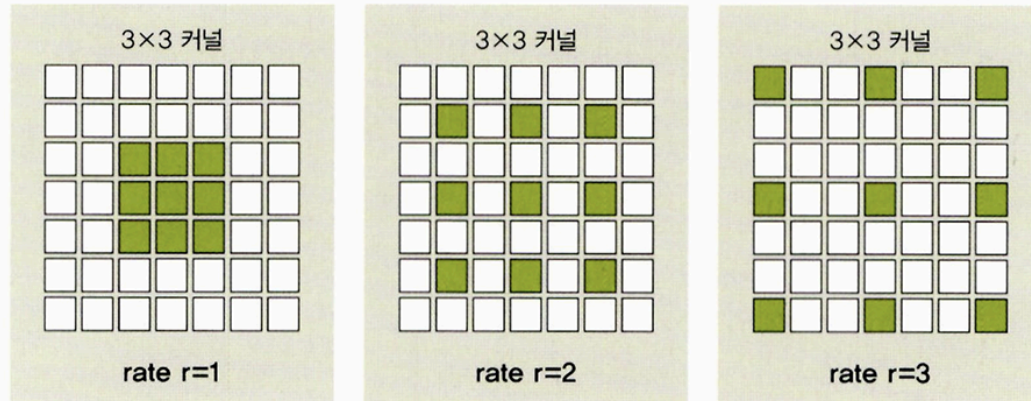
▼ 그림 6-54 DeepLab의 인코더-디코더 구조



- Atrous 합성곱
 - 필터 내부에 빈 공간을 둔 채 작동

- 얼마나 많은 빈 공간을 가질지 결정하는 파라미터로 rate가 있는데 rate r 값이 커질수록 빈 공간은 더 많아짐

▼ 그림 6-55 Atrous 합성곱



- 이미지 분할에서 중요한 수용 영역은 Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않고도 수용 영역을 키울 수 있음.
- 일반 CNN은 1/32로 출력이 줄어들지만, Atrous 합성곱은 1/8로 줄어들기에 특성 맵 크기가 기존보다 4배 보존됨

▼ 그림 6-56 Atrous 합성곱 효과

